TP02 - Relacionamento N:N

Alunos participantes: Isabel Cristina, Laura Bargas, Pedro Mattar e Yuri Penido.

O Busca de Cursos (TP02) é uma evolução do sistema acadêmico para gestão de cursos livres da PUC Minas. Esta versão implementa o Relacionamento N:N, permitindo que usuários se inscrevam em múltiplos cursos de outros proponentes. O sistema utiliza persistência em arquivos binários de acesso aleatório (RandomAccessFile) e estruturas de dados avançadas como Tabelas Hash Extensível para buscas diretas e Árvores B+ para indexação e ordenação.

Estrutura de Classes Criadas

- **Inscricao:** Entidade que representa o relacionamento N:N, armazenando os atributos idInscricao, idUsuario (aluno), idCurso e a dataInscricao.

```java
public class Inscricao implements InterfaceEntidade {

    private int idInscricao;
    private int idCurso;
    private int idUsuario;
    private int dataInscricaoEpoch; // epoch day

    public Inscricao() {
        this(-1, -1, -1, (int)java.time.LocalDate.now().toEpochDay());
    }

    public Inscricao(int idCurso, int idUsuario) {
        this(-1, idCurso, idUsuario, (int)java.time.LocalDate.now().toEpochDay());
    }

    public Inscricao(int id, int idCurso, int idUsuario, int dataEpoch) {
        this.idInscricao = id;
        this.idCurso = idCurso;
        this.idUsuario = idUsuario;
        this.dataInscricaoEpoch = dataEpoch;
    }

    public int getID() { return idInscricao; }
    public void setID(int id) { this.idInscricao = id; }

    public int getIdCurso() { return idCurso; }
    public void setIdCurso(int idCurso) { this.idCurso = idCurso; }

    public int getIdUsuario() { return idUsuario; }
    public void setIdUsuario(int idUsuario) { this.idUsuario = idUsuario; }
```

- **ArquivoInscricao:** Classe responsável pelo CRUD das inscrições, gerenciando duas novas Árvores B+ para indexar o relacionamento: uma por idUsuario e outra por idCurso.

```java
public class ArquivoInscricao extends Arquivo<Inscricao> {

    ArvoreBMais<ParIdId> indiceCursoInscricao; // (idCurso; idInscricao)
    ArvoreBMais<ParIdId> indiceUsuarioInscricao; // (idUsuario; idInscricao)

    public ArquivoInscricao() throws Exception {
        super("inscricao", Inscricao.class.getConstructor());
        indiceCursoInscricao = new ArvoreBMais<>(
            ParIdId.class.getConstructor(),
            4, "./dados/inscricao/indiceCursoInscricao.db");
        indiceUsuarioInscricao = new ArvoreBMais<>(
            ParIdId.class.getConstructor(),
            4, "./dados/inscricao/indiceUsuarioInscricao.db");
    }

    @Override
    public int create(Inscricao ins) throws Exception {
        int id = super.create(ins);
        indiceCursoInscricao.create(new ParIdId(ins.getIdCurso(), id));
        indiceUsuarioInscricao.create(new ParIdId(ins.getIdUsuario(), id));
        return id;
    }

    public Inscricao[] readByCurso(int idCurso) throws Exception {
        ArrayList<ParIdId> p = indiceCursoInscricao.read(new ParIdId(idCurso, -1));
        if(p.isEmpty()) return new Inscricao[0];
        Inscricao[] resultado = new Inscricao[p.size()];
        int i=0;
        for(ParIdId par : p) {
            resultado[i++] = super.read(par.getId2());
```

- **ControleInscricao:** Gerencia a lógica de negócio das matrículas, permitindo que o usuário busque cursos, realize inscrições e cancele participações.

```java
public void menuInscricoes(int userId) throws Exception {
    do {
        if(minhas.length==0) {
            System.out.println("Nenhuma inscricao encontrada.");
        } else {
            for(int i=0;i<minhas.length;i++) {
                entidades.inscricoes.Inscricao ins = minhas[i];
                Curso c = arqCursos.read(ins.getIdCurso());
                if(c!=null) {
                    String status = "";
                    if(c.getEstado()==1) status = " (INSCRIÇÕES ENCERRADAS)";
                    else if(c.getEstado()==2) status = " (CONCLUIDO)";
                    else if(c.getEstado()==3) status = " (CANCELADO)";
                    System.out.println("(" + (i+1) + ") " + c.getNome() + " - " + c.getDataInicio() + s
                }
            }
        }

        System.out.println();
        System.out.println("(A) Buscar curso por codigo");
        System.out.println("(B) Buscar curso por palavras-chave");
        System.out.println("(C) Listar todos os cursos");
        System.out.println();
        System.out.println("(R) Retornar ao menu anterior");
        System.out.print("\nOpcao: ");
        String op = console.nextLine();
        if(op.length()==0) continue;
        char ch = op.charAt(0);

        if(ch=='A' || ch=='a') {
            String codigo = v.pedirCodigo(console);
```

- **VisaoInscricao:** Interface de terminal para o menu de inscrições, responsável por exibir a lista de cursos em que o usuário está matriculado e a tela de detalhes para novas adesões.

```java
import java.util.Scanner;
import entidades.cursos.Curso;

public class VisaoInscricao {

    public String pedirCodigo(Scanner console) {
        System.out.print(s: "Codigo do curso: ");
        return console.nextLine().trim();
    }

    public void mostraCursoDetalhado(Curso c, String autorNome) {
        System.out.println("CÓDIGO........: " + c.getCodigoCompartilhavel());
        System.out.println("CURSO.........: " + c.getNome());
        System.out.println("AUTOR.........: " + (autorNome==null?"":autorNome));
        System.out.println("DESCRIÇÃO.....: " + c.getDescricao());
        System.out.println("DATA DE INÍCIO: " + c.getDataInicio());
        System.out.println();
    }

}
```

- **ParIdId:** Classe de suporte que define o par de IDs utilizado nas Árvores B+ para vincular usuários e cursos de forma eficiente.

```java
public short size() {
  return this.TAMANHO;
}

public int compareTo(ParIdId a) {
  if(this.id1 != a.id1)
    return this.id1 - a.id1;
  else {
    if(this.id2 == -1)
      return 0;
    else
      return this.id2 - a.id2;
  }
}

public String toString() {
  return this.id1 + ";" + this.id2;
}

public byte[] toByteArray() throws IOException {
  ByteArrayOutputStream baos = new ByteArrayOutputStream();
  DataOutputStream dos = new DataOutputStream(baos);
  dos.writeInt(this.id1);
  dos.writeInt(this.id2);
  return baos.toByteArray();
}
```

- **ParCodigoId:** Classe de suporte para o índice de busca rápida, vinculando o código alfanumérico (NanoID) ao ID do curso.

```java
public ParCodigoId(String codigo, int id) throws Exception {
    if(codigo.getBytes().length>10)
        throw new Exception(message: "Código inválido");
    this.codigo = codigo;
    this.id = id;
}

public String getCodigo() { return codigo; }
public int getId() { return id; }

@Override
public int hashCode() {
    return Math.abs(this.codigo.hashCode());
}

public short size() {
    return this.TAMANHO;
}

public String toString() {
    return "("+this.codigo + ";" + this.id+")";
}

public byte[] toByteArray() throws IOException {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
```

Operações Especiais Implementadas:

1. **Exclusão Lógica e Reuso:** O sistema não deleta fisicamente o registro de imediato; ele marca com uma "lápide" e insere o endereço na lista de espaços disponíveis.

```java
public void insereEspacoVazio(long endereco, short tamanho) throws Exception {
    arquivo.seek(pos: 4);   // posição da cabeça da lista no arquivo
    long end = arquivo.readLong();   // endereço do primeiro elemento da lista

    // lista vazia
    if(end == -1) {
        arquivo.seek(pos: 4);
        arquivo.writeLong(endereco);
        arquivo.seek(endereco+3);
        arquivo.writeLong(-1);
    }

    // insere na lista existente, de forma ordenada
    else {
        long endAnterior = 4;
        while(end!=-1) {
            arquivo.seek(end+1);
            short tam = arquivo.readShort();
            if(tamanho<tam)
                break;
            endAnterior = end;
            end = arquivo.readLong();
        }
        arquivo.seek(endereco+3);
        arquivo.writeLong(end);
        if(endAnterior==4)
            arquivo.seek(pos: 4);
        else
            arquivo.seek(endAnterior+3);
        arquivo.writeLong(endereco);
```

2. **Gestão de Inscritos:** O proponente do curso pode visualizar uma lista de inscritos e exportá-la para o formato **CSV**, facilitando a gestão externa de alunos.

```
(1) Pedro Mattar (2026-05-12)
(2) yuri (2026-05-12)

(A) Exportar lista

(R) Retornar ao menu anterior

Opcao:
```

3. **Sincronização de Índices:** Ao criar uma inscrição, o sistema atualiza simultaneamente os índices diretos e as árvores de relacionamento para garantir que a visão de "Minhas Inscrições" seja imediata.

```java
public class ArquivoInscricao extends Arquivo<Inscricao> {

    ArvoreBMais<ParIdId> indiceCursoInscricao; // (idCurso; idInscricao)
    ArvoreBMais<ParIdId> indiceUsuarioInscricao; // (idUsuario; idInscricao)

    public ArquivoInscricao() throws Exception {
        super(nomeEntidade: "inscricao", Inscricao.class.getConstructor());
        indiceCursoInscricao = new ArvoreBMais<>(
            ParIdId.class.getConstructor(),
            o: 4, na: "./dados/inscricao/indiceCursoInscricao.db");
        indiceUsuarioInscricao = new ArvoreBMais<>(
            ParIdId.class.getConstructor(),
            o: 4, na: "./dados/inscricao/indiceUsuarioInscricao.db");
    }

    @Override
    public int create(Inscricao ins) throws Exception {
        int id = super.create(ins);
        indiceCursoInscricao.create(new ParIdId(ins.getIdCurso(), id));
        indiceUsuarioInscricao.create(new ParIdId(ins.getIdUsuario(), id));
        return id;
    }
```

Checklist:

1. Há um CRUD da entidade de associação CursoUsuario (que estende a classe ArquivoIndexado, adicionando Tabelas Hash Extensível e Árvores B+ como índices diretos e indiretos conforme necessidade) que funciona corretamente?

   Sim.

2. A visão de inscrições está corretamente implementada e permite consultas aos cursos em que um usuário está inscrito?

   Sim.

3. A visão de cursos funciona corretamente e permite a gestão dos usuários inscritos em um curso?

   Sim.

4. Há uma visualização dos cursos de outras pessoas por meio de um código NanoID?

   Sim.

5. A integridade do relacionamento entre cursos e usuários está mantida em todas as operações?

   Sim, operações de exclusão de curso ou usuário tratam as dependências na tabela de associação.

6. O trabalho compila corretamente?

   Sim.

7. O trabalho está completo e funcionando sem erros de execução?

   Sim.

8. O trabalho é original e não a cópia de um trabalho de outro grupo?

   Sim.